

计算机与信息工程学院实验报告

姓名	—	学号	
专业		年级	
课程	操作系统	主讲教师	刘扬
实验时间（年月日时）	2025 年 11 月 13 日 10-12 时		
实验地点	计算机大楼 313	辅导教师	刘扬
实验题目	存储管理模拟		
实验目的	存储管理的主要功能之一是合理地分配空间。请求页式管理是一种常用的虚拟存储管理技术。 本实验的目的是通过请求页式存储管理中页面置换算法模拟设计，了解虚拟存储技术的技术特点，掌握请求页式存储管理的页面置换算法。		
实验环境（硬件和软件）	Ubuntu + CPP		

一、OS 课程实验

实验内容：

- 1、设计一个虚拟存储区和内存工作区,并使用下列算法计算访问命中率.
- 进先出的算法（FIFO）
- 最近最少使用的算法（LRU）最佳淘汰算法(OPT)
- 最少访问页面算法(LFU)
- 最近最不经常使用算法(NUR)
- 命中率=(1-页面失效次数)/页地址流长度

实验步骤：

1、宏定义和结构

```
#define TRUE 1
#define FALSE 0
#define INVALID -1
#define total_instruction 320
#define total_vp 32
#define clear_period 50

typedef struct {
```

```

    int pn;
    int pfn;
    int counter;
    int time;
}pl_type;

pl_type pl[total_vp];

typedef struct pfc_struct {
    int pn;
    int pfn;
    struct pfc_struct *next;
}pfc_type;

pfc_type pfc[total_vp], *freepf_head, *busypf_head, *busypf_tail;

```

2、函数声明

```

int initialize(int);
int FIFO(int);
int LRU(int);
int LFU(int);
int NUR(int);
int OPT(int);

```

3、变量定义

```

int diseffect, a[total_instruction];
int page[total_instruction], offset[total_instruction];

```

4、main 函数

```

int main() {
    int s, i, j;
    srand((unsigned)time(NULL));
    s = (float)319*rand()/32767/32767/2+1;
    for (int i = 0; i < total_instruction; i += 4) {
        if (s < 0 || s > 319) {
            printf("When i == %d, Error, s == %d\n", i, s);
            exit(0);
        }

        a[i] = s;
        a[i+1] = a[i] + 1;
        a[i+2] = (float)a[i] * rand()/32767/32767/2;
        a[i+3] = a[i+2] + 1;
    }
}

```

```

        s = (float)(318 - a[i+2]) * rand()/32767/32767/2 + a[i+2] + 2;
        if ((a[i+2] > 318) || (s > 319)) {
            printf("a[%d+2], a number which is: %d and s == %d\n", i, a[i+2], s);
        }
    }

    for (int i = 0; i < total_instruction; i++) {
        page[i] = a[i] / 10;
        offset[i] = a[i] % 10;
    }

    for (int i = 4; i <= 32; i++) {
        printf("%2d page frames | ", i);
        FIFO(i);
        LRU(i);
        OPT(i);
        LFU(i);
        NUR(i);
    }

    return 0;
}

```

5、初始化函数

```

int initialize(int total_pf) {
    int i;
    diseffect = 0;
    for (int i = 0; i < total_vp; i++) {
        pl[i].pfn = INVALID;
        pl[i].counter = 0;
        pl[i].time = -1;
    }

    for (int i = 0; i < total_pf - 1; i++) {
        pfc[i].next = &pfc[i+1];
        pfc[i].pfn = i;
    }

    pfc[total_pf - 1].next = NULL;
    pfc[total_pf - 1].pfn = total_pf - 1;
    freepf_head = &pfc[0];
    return 0;
}

```

6、FIFO 函数

```

int FIFO(int total_pf) {
    int i;
    pfc_type *p;
    initialize(total_pf);
    busypf_head = busypf_tail = NULL;
    for (i = 0; i < total_instruction; i++) {
        if (pl[page[i]].pfn == INVALID) {
            diseffect++;
            if (freepf_head == NULL) {
                /* 没有空闲帧，回收 busy 列表头部 */
                if (busypf_head != NULL) {
                    p = busypf_head->next;
                    pl[busypf_head->pn].pfn = INVALID;
                    freepf_head = busypf_head;
                    freepf_head->next = NULL;
                    busypf_head = p;
                    if (busypf_head == NULL) busypf_tail = NULL;
                } else {
                    /* 不应到达：既无空闲帧也无占用帧，保护性退出 */
                    fprintf(stderr, "FIFO error: no frames available\n");
                    exit(1);
                }
            }
            p = freepf_head->next;
            freepf_head->pn = page[i];
            pl[page[i]].pfn = freepf_head->pfn;
            freepf_head->next = NULL;
            if (busypf_tail == NULL) {
                busypf_tail = busypf_head = freepf_head;
            } else {
                busypf_tail->next = freepf_head;
                busypf_tail = freepf_head;
            }
            freepf_head = p;
        }
    }
    printf("FIFO:%6.4f | ", 1 - (float)diseffect/total_instruction);
    return 0;
}

```

7、LRU 函数

```

int LRU(int total_pf) {
    int min, minj, i, j, present_time;
    pfc_type *p;

```

```

initialize(total_pf);
busypf_head = busypf_tail = NULL;
present_time = 0;
for (int i = 0; i < total_instruction; i++) {
    /* 每次访问都增加时间戳，避免多个页面拥有相同时间 */
    if (pl[page[i]].pfn == INVALID) {
        diseffect++;
        if (freepf_head == NULL) {
            min = 32767;
            minj = -1;
            for (j = 0; j < total_vp; j++) {
                if (pl[j].pfn != INVALID && pl[j].time < min) {
                    min = pl[j].time;
                    minj = j;
                }
            }
            if (minj == -1) {
                /* 保护性处理：没有找到可淘汰页（不应发生） */
                fprintf(stderr, "LRU error: no victim found\n");
                exit(1);
            }
            /* 从 busy 列表中摘除被淘汰的节点，作为新的
freepf_head */
            pfc_type *target = &pfc[pl[minj].pfn];
            pfc_type *prev = NULL;
            pfc_type *cur = busypf_head;
            while (cur != NULL && cur != target) {
                prev = cur; cur = cur->next;
            }
            if (cur == NULL) {
                /* 如果没在 busy 列表中找到，直接使用 target */
                freepf_head = target;
            } else {
                /* 从 busy 列表中摘除 cur */
                if (prev == NULL) busypf_head = cur->next;
                else prev->next = cur->next;
                if (cur == busypf_tail) busypf_tail = prev;
                freepf_head = cur;
            }
            pl[minj].pfn = INVALID;
            pl[minj].time = 0;
            freepf_head->next = NULL;
        }
        /* 使用 freepf_head 分配帧并加入 busy 列表 */

```

```

        p = freepf_head->next;
        freepf_head->pn = page[i];
        pl[page[i]].pfn = freepf_head->pfn;
        pl[page[i]].time = present_time++;
        freepf_head->next = NULL;
        if (busypf_tail == NULL) {
            busypf_tail = busypf_head = freepf_head;
        } else {
            busypf_tail->next = freepf_head;
            busypf_tail = freepf_head;
        }
        freepf_head = p;
    } else {
        pl[page[i]].time = present_time++;
    }
}
printf("LRU:%6.4f | ", 1 - (float)diseffect/total_instruction);
return 0;
}

```

8、OPT 函数

```

int OPT(int total_pf) {
    int i, j, max, maxpage, d, dist[total_vp];
    pfc_type *t;
    initialize(total_pf);
    for (i = 0; i < total_instruction; i++) {
        if (pl[page[i]].pfn == INVALID) {
            diseffect++;
            if (freepf_head == NULL) {
                /* 计算每个驻留页到下一次使用的距离，选取距离最大的
页淘汰 */
                for (j = 0; j < total_vp; j++) {
                    if (pl[j].pfn != INVALID) {
                        dist[j] = 32767; /* 默认表示在未来不会再用 */
                    } else {
                        dist[j] = -1; /* 不在内存中 */
                    }
                }
                for (j = 0; j < total_vp; j++) {
                    if (pl[j].pfn != INVALID) {
                        int k;
                        dist[j] = 32767;
                        for (k = i + 1; k < total_instruction; k++) {
                            if (page[k] == j) { dist[j] = k - i; break; }
                        }
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    }
    max = -1; maxpage = -1;
    for (j = 0; j < total_vp; j++) {
        if (pl[j].pfn != INVALID) {
            if (dist[j] == 32767) { maxpage = j; break; } /* 永远不
会再用，立即淘汰 */
            if (dist[j] > max) { max = dist[j]; maxpage = j; }
        }
    }
    if (maxpage >= 0) {
        freepf_head = &pfc[pl[maxpage].pfn];
        freepf_head->next = NULL;
        pl[maxpage].pfn = INVALID;
    }
    }
    pl[page[i]].pfn = freepf_head->pfn;
    freepf_head = freepf_head->next;
}
}
printf("OPT:%6.4f | ", 1 - (float)diseffect/total_instruction);
return 0;
}

```

9、LFU 函数

```

int LFU(int total_pf) {
    int i, j, min, minpage;
    pfc_type *t;
    initialize(total_pf);
    for (i = 0; i < total_instruction; i++) {
        if (pl[page[i]].pfn == INVALID) {
            diseffect++;
            if (freepf_head == NULL) {
                min = 32767;
                for (j = 0; j < total_vp; j++) {
                    if (min > pl[j].counter && pl[j].pfn != INVALID) {
                        min = pl[j].counter;
                        minpage = j;
                    }
                }
            }
            freepf_head = &pfc[pl[minpage].pfn];
            pl[minpage].pfn = INVALID;
            pl[minpage].counter = 0;
        }
    }
}

```

```

        freepf_head->next = NULL;
    }
    pl[page[i]].pfn = freepf_head->pfn;
    pl[page[i]].counter++;
    freepf_head = freepf_head->next;
} else {
    pl[page[i]].counter = pl[page[i]].counter + 1;
}
}
printf("LFU:%6.4f | ", 1 - (float)diseffect/total_instruction);
return 0;
}

```

10、NUR 函数

```

int NUR(int total_pf) {
    int i, j, dp, cont_flag, old_dp;
    pfc_type *t;
    initialize(total_pf);
    dp = 0;

    for (i = 0; i < total_instruction; i++) {
        if (pl[page[i]].pfn == INVALID) {
            diseffect++;
            if (freepf_head == NULL) {
                cont_flag = TRUE;
                old_dp = dp;
                while(cont_flag) {
                    if(pl[dp].counter == 0 && pl[dp].pfn != INVALID) {
                        cont_flag = FALSE;
                    } else {
                        dp++;
                        if(dp == total_vp) {
                            dp = 0;
                        }
                        if (dp == old_dp) {
                            for (j = 0; j < total_vp; j++) {
                                pl[j].counter = 0;
                            }
                        }
                    }
                }
            }
            freepf_head = &pfc[pl[dp].pfn];
            pl[dp].pfn = INVALID;
            freepf_head->next = NULL;
        }
    }
}

```

```

    }
    pl[page[i]].pfn = freepf_head->pfn;
    freepf_head->pn = page[i];
    freepf_head = freepf_head->next;
} else {
    pl[page[i]].counter = 1;
}
if (i % clear_period == 0) {
    for (j = 0; j < total_vp; j++) {
        pl[j].counter = 0;
    }
}
}
printf("NUR:%6.4f\n", 1 - (float)diseffect/total_instruction);
return 0;
}

```

11、运行程序，记录程序运行结果，debug。

实验数据记录：

1、第一次运行结果

```

$ ./memory2
4 page frames | FIFO:0.5312 | LRU:0.5344 | OPT:0.6531 | LFU:0.5125 | NUR:0.5469
5 page frames | FIFO:0.5531 | LRU:0.5531 | OPT:0.6812 | LFU:0.5250 | NUR:0.5562
6 page frames | FIFO:0.5875 | LRU:0.5813 | OPT:0.7063 | LFU:0.5469 | NUR:0.5781
7 page frames | FIFO:0.5906 | LRU:0.5938 | OPT:0.7250 | LFU:0.5531 | NUR:0.5969
8 page frames | FIFO:0.6062 | LRU:0.6000 | OPT:0.7437 | LFU:0.5813 | NUR:0.6000
9 page frames | FIFO:0.6219 | LRU:0.6250 | OPT:0.7594 | LFU:0.5969 | NUR:0.6031
10 page frames | FIFO:0.6406 | LRU:0.6344 | OPT:0.7750 | LFU:0.6156 | NUR:0.6375
11 page frames | FIFO:0.6594 | LRU:0.6594 | OPT:0.7875 | LFU:0.6313 | NUR:0.6719
12 page frames | FIFO:0.6656 | LRU:0.6781 | OPT:0.8000 | LFU:0.6344 | NUR:0.6656
13 page frames | FIFO:0.6781 | LRU:0.6906 | OPT:0.8125 | LFU:0.6500 | NUR:0.6781
14 page frames | FIFO:0.6844 | LRU:0.7000 | OPT:0.8250 | LFU:0.6719 | NUR:0.6875
15 page frames | FIFO:0.7156 | LRU:0.7094 | OPT:0.8344 | LFU:0.6969 | NUR:0.7250
16 page frames | FIFO:0.7281 | LRU:0.7312 | OPT:0.8438 | LFU:0.7094 | NUR:0.7125
17 page frames | FIFO:0.7406 | LRU:0.7437 | OPT:0.8500 | LFU:0.7250 | NUR:0.7375
18 page frames | FIFO:0.7500 | LRU:0.7531 | OPT:0.8562 | LFU:0.7406 | NUR:0.7531
19 page frames | FIFO:0.7531 | LRU:0.7625 | OPT:0.8625 | LFU:0.7594 | NUR:0.7563
20 page frames | FIFO:0.7781 | LRU:0.7812 | OPT:0.8687 | LFU:0.7688 | NUR:0.7875
21 page frames | FIFO:0.7781 | LRU:0.7969 | OPT:0.8719 | LFU:0.7844 | NUR:0.8094
22 page frames | FIFO:0.7875 | LRU:0.8062 | OPT:0.8750 | LFU:0.7969 | NUR:0.8281
23 page frames | FIFO:0.7969 | LRU:0.8125 | OPT:0.8781 | LFU:0.8000 | NUR:0.8156
24 page frames | FIFO:0.8000 | LRU:0.8313 | OPT:0.8813 | LFU:0.8094 | NUR:0.8344
25 page frames | FIFO:0.8500 | LRU:0.8375 | OPT:0.8844 | LFU:0.8219 | NUR:0.8469
26 page frames | FIFO:0.8562 | LRU:0.8375 | OPT:0.8875 | LFU:0.8469 | NUR:0.8562
27 page frames | FIFO:0.8562 | LRU:0.8531 | OPT:0.8906 | LFU:0.8594 | NUR:0.8687
28 page frames | FIFO:0.8750 | LRU:0.8594 | OPT:0.8938 | LFU:0.8625 | NUR:0.8719
29 page frames | FIFO:0.8813 | LRU:0.8750 | OPT:0.8969 | LFU:0.8750 | NUR:0.8844
30 page frames | FIFO:0.8813 | LRU:0.8875 | OPT:0.9000 | LFU:0.8938 | NUR:0.8938
31 page frames | FIFO:0.8844 | LRU:0.9000 | OPT:0.9000 | LFU:0.8938 | NUR:0.9000
32 page frames | FIFO:0.9000 | LRU:0.9000 | OPT:0.9000 | LFU:0.9000 | NUR:0.9000

```

2、第二次运行结果

4	page frames	FIFO:0.5031	LRU:0.5031	OPT:0.6313	LFU:0.5188	NUR:0.4969
5	page frames	FIFO:0.5312	LRU:0.5281	OPT:0.6625	LFU:0.5437	NUR:0.5344
6	page frames	FIFO:0.5594	LRU:0.5594	OPT:0.6906	LFU:0.5656	NUR:0.5500
7	page frames	FIFO:0.5719	LRU:0.5719	OPT:0.7094	LFU:0.5781	NUR:0.5750
8	page frames	FIFO:0.5750	LRU:0.5813	OPT:0.7281	LFU:0.5844	NUR:0.5938
9	page frames	FIFO:0.6031	LRU:0.6000	OPT:0.7469	LFU:0.6125	NUR:0.6125
10	page frames	FIFO:0.6094	LRU:0.6125	OPT:0.7656	LFU:0.6375	NUR:0.6156
11	page frames	FIFO:0.6187	LRU:0.6219	OPT:0.7812	LFU:0.6469	NUR:0.6281
12	page frames	FIFO:0.6281	LRU:0.6406	OPT:0.7969	LFU:0.6594	NUR:0.6594
13	page frames	FIFO:0.6469	LRU:0.6625	OPT:0.8125	LFU:0.6844	NUR:0.6719
14	page frames	FIFO:0.6906	LRU:0.6812	OPT:0.8219	LFU:0.7094	NUR:0.6875
15	page frames	FIFO:0.7000	LRU:0.6969	OPT:0.8281	LFU:0.7219	NUR:0.7188
16	page frames	FIFO:0.7063	LRU:0.7094	OPT:0.8344	LFU:0.7281	NUR:0.7094
17	page frames	FIFO:0.7344	LRU:0.7312	OPT:0.8406	LFU:0.7437	NUR:0.7531
18	page frames	FIFO:0.7500	LRU:0.7469	OPT:0.8469	LFU:0.7563	NUR:0.7437
19	page frames	FIFO:0.7563	LRU:0.7719	OPT:0.8531	LFU:0.7625	NUR:0.7719
20	page frames	FIFO:0.7719	LRU:0.7844	OPT:0.8594	LFU:0.7656	NUR:0.7688
21	page frames	FIFO:0.7781	LRU:0.7969	OPT:0.8656	LFU:0.7812	NUR:0.7875
22	page frames	FIFO:0.7812	LRU:0.8062	OPT:0.8719	LFU:0.8031	NUR:0.7844
23	page frames	FIFO:0.8250	LRU:0.8188	OPT:0.8781	LFU:0.8062	NUR:0.7937
24	page frames	FIFO:0.8562	LRU:0.8281	OPT:0.8844	LFU:0.8219	NUR:0.8094
25	page frames	FIFO:0.8562	LRU:0.8344	OPT:0.8906	LFU:0.8438	NUR:0.8375
26	page frames	FIFO:0.8562	LRU:0.8469	OPT:0.8938	LFU:0.8469	NUR:0.8594
27	page frames	FIFO:0.8719	LRU:0.8562	OPT:0.8969	LFU:0.8531	NUR:0.8656
28	page frames	FIFO:0.8781	LRU:0.8719	OPT:0.9000	LFU:0.8656	NUR:0.8719
29	page frames	FIFO:0.8969	LRU:0.8781	OPT:0.9031	LFU:0.8875	NUR:0.8906
30	page frames	FIFO:0.9000	LRU:0.8938	OPT:0.9031	LFU:0.8969	NUR:0.9000
31	page frames	FIFO:0.9031	LRU:0.9031	OPT:0.9031	LFU:0.9031	NUR:0.9031
32	page frames	FIFO:0.9031	LRU:0.9031	OPT:0.9031	LFU:0.9031	NUR:0.9031

3、第三次运行结果

4	page frames	FIFO:0.5406	LRU:0.5375	OPT:0.6438	LFU:0.5312	NUR:0.5437
5	page frames	FIFO:0.5625	LRU:0.5625	OPT:0.6719	LFU:0.5500	NUR:0.5656
6	page frames	FIFO:0.5844	LRU:0.5813	OPT:0.6969	LFU:0.5813	NUR:0.5688
7	page frames	FIFO:0.5938	LRU:0.6000	OPT:0.7156	LFU:0.6000	NUR:0.5938
8	page frames	FIFO:0.5969	LRU:0.6062	OPT:0.7344	LFU:0.6125	NUR:0.6094
9	page frames	FIFO:0.6031	LRU:0.6219	OPT:0.7531	LFU:0.6281	NUR:0.6187
10	page frames	FIFO:0.6156	LRU:0.6375	OPT:0.7719	LFU:0.6344	NUR:0.6406
11	page frames	FIFO:0.6562	LRU:0.6469	OPT:0.7875	LFU:0.6531	NUR:0.6375
12	page frames	FIFO:0.6687	LRU:0.6625	OPT:0.8000	LFU:0.6750	NUR:0.6656
13	page frames	FIFO:0.6812	LRU:0.6844	OPT:0.8125	LFU:0.6875	NUR:0.6750
14	page frames	FIFO:0.6875	LRU:0.6969	OPT:0.8219	LFU:0.7063	NUR:0.7000
15	page frames	FIFO:0.7156	LRU:0.7094	OPT:0.8313	LFU:0.7188	NUR:0.7063
16	page frames	FIFO:0.7344	LRU:0.7250	OPT:0.8406	LFU:0.7312	NUR:0.7312
17	page frames	FIFO:0.7344	LRU:0.7437	OPT:0.8469	LFU:0.7312	NUR:0.7375
18	page frames	FIFO:0.7469	LRU:0.7594	OPT:0.8531	LFU:0.7688	NUR:0.7344
19	page frames	FIFO:0.7625	LRU:0.7844	OPT:0.8594	LFU:0.7844	NUR:0.7656
20	page frames	FIFO:0.7688	LRU:0.7844	OPT:0.8656	LFU:0.8000	NUR:0.7875
21	page frames	FIFO:0.7875	LRU:0.7937	OPT:0.8719	LFU:0.8125	NUR:0.8125
22	page frames	FIFO:0.7969	LRU:0.8000	OPT:0.8781	LFU:0.8281	NUR:0.8313
23	page frames	FIFO:0.8250	LRU:0.8188	OPT:0.8844	LFU:0.8375	NUR:0.8344
24	page frames	FIFO:0.8562	LRU:0.8281	OPT:0.8906	LFU:0.8406	NUR:0.8531
25	page frames	FIFO:0.8562	LRU:0.8406	OPT:0.8938	LFU:0.8531	NUR:0.8813
26	page frames	FIFO:0.8562	LRU:0.8656	OPT:0.8969	LFU:0.8562	NUR:0.8687
27	page frames	FIFO:0.8594	LRU:0.8781	OPT:0.9000	LFU:0.8813	NUR:0.8781
28	page frames	FIFO:0.8687	LRU:0.8875	OPT:0.9000	LFU:0.8844	NUR:0.8719
29	page frames	FIFO:0.8875	LRU:0.8969	OPT:0.9000	LFU:0.8938	NUR:0.8844
30	page frames	FIFO:0.8875	LRU:0.8969	OPT:0.9000	LFU:0.8938	NUR:0.8875
31	page frames	FIFO:0.9000	LRU:0.9000	OPT:0.9000	LFU:0.9000	NUR:0.8938
32	page frames	FIFO:0.9000	LRU:0.9000	OPT:0.9000	LFU:0.9000	NUR:0.9000

问题讨论：

1、理论上 OPT 算法因预知未来访问序列，应始终具有最高命中率。然而，实际实验中 FIFO 在多个帧数下命中率超过 OPT，这可能是因为特定访问序列的模式与 OPT 的决策规则产生了不利交互。具体原因可能有：未来访问距离相同时的替换策略：当多个页在未来都不再访问或具有相同距离时，OPT 的裁决规则可能导致其替换了后续仍频繁需要的页；局部性模式与 FIFO 的巧合匹配：在周期性或循环访问序列中，FIFO 的先进先出顺序可能偶然保留了即将访问的页，而 OPT 因“过于理性”地保留近期使用但未来最远的页，反而提前将其换出；有限序列长度下的边界效应：序列较短时，一次非最优替换可能引发连锁反应，放大 FIFO 的偶然优势；